

RLHF with PPO & the KL Penalty.

With a reward model in hand, we optimize the policy to *earn* reward — but on a leash. A KL penalty to the reference policy stops the model from drifting into nonsense that merely fools the reward model. We derive the shaped reward, the KL term, and PPO’s clipped surrogate, with every number computed.

THE EQUATION

$$\max_{\theta} \mathbb{E}_{y \sim \pi_{\theta}(\cdot | x)} [r(x, y)] - \beta \text{KL}(\pi_{\theta}(\cdot | x) \| \pi_{\text{ref}}(\cdot | x))$$

Worked example. A 3-token vocabulary with explicit π_{θ} and π_{ref} : compute the KL, sweep β to see the effective reward shrink, and run one PPO clip step for both signs of advantage.

Topic 2 of 3 in this module · companion to the course page · v1.0

Contents

1	The KL-regularised RLHF objective	2
2	Why the KL term prevents reward hacking	3
3	Worked example: KL on a 3-token vocabulary	3
4	Heatmap: π_θ vs. π_{ref}	4
5	PPO's clipped surrogate objective	4
6	Properties that matter	5
7	Where this sits in the track	6
A	Reproducible (NumPy)	6

1 The KL-regularised RLHF objective

The policy π_θ generates a response y from prompt x and is rewarded by the frozen reward model $r(x, y)$ from Topic 1. Maximising reward alone is dangerous: the policy can exploit blind spots of r . We therefore add a penalty that keeps π_θ close to the SFT reference π_{ref} :

$$J(\theta) = \mathbb{E}_x \mathbb{E}_{y \sim \pi_\theta(\cdot | x)} [r(x, y)] - \beta \text{KL}(\pi_\theta(\cdot | x) \| \pi_{\text{ref}}(\cdot | x)) \quad (1)$$

The coefficient $\beta > 0$ trades reward against fidelity. As $\beta \rightarrow \infty$ the policy is pinned to π_{ref} ; as $\beta \rightarrow 0$ it chases reward unrestrained.

Symbol	Meaning	Shape
$\pi_\theta(y x)$	current (trainable) policy	distribution over y
$\pi_{\text{ref}}(y x)$	frozen SFT reference	distribution over y
$r(x, y)$	frozen reward-model score	$\mathbb{R}$
β	KL penalty coefficient	$\mathbb{R}_{>0}$
$\text{KL}(p \ q)$	$\sum_v p(v) \log \frac{p(v)}{q(v)}$	$\mathbb{R}_{\geq 0}$
A	advantage estimate	$\mathbb{R}$
ρ	probability ratio $\pi_\theta / \pi_{\text{old}}$	$\mathbb{R}_{>0}$

The reward model is a *learned, imperfect* judge. Without a leash, the policy behaves like a student who, instead of writing good essays, learns the exact phrases that flatter *this one grader*. The KL term says: “you may improve, but stay recognisably like the model the grader was trained on.”

2 Why the KL term prevents reward hacking

The reward model was fit on samples from π_{ref} (Topic 1). It is trustworthy only on that distribution; far from it, $r(x, y)$ is extrapolation and may be wildly over-optimistic. The KL penalty bounds how far π_θ can move, so generated y stay in-distribution for r , keeping the reward signal meaningful. Empirically, dropping β too low produces high *measured* reward but degenerate, repetitive, or off-topic text — the signature of reward hacking.

2.1 Per-token KL and the shaped reward

The sequence-level KL decomposes over tokens. For a single decoding step with current-token distribution $\pi_\theta(\cdot | s)$ and reference $\pi_{\text{ref}}(\cdot | s)$ at state s ,

$$\text{KL}(\pi_\theta(\cdot | s) \| \pi_{\text{ref}}(\cdot | s)) = \sum_v \pi_\theta(v | s) \log \frac{\pi_\theta(v | s)}{\pi_{\text{ref}}(v | s)}.$$

In practice PPO folds the penalty into the reward as a *per-token shaped reward*: for the token y_t actually sampled, the environment returns

$$R_t = \underbrace{r(x, y) [t = T]}_{\text{terminal RM score}} - \beta \log \frac{\pi_\theta(y_t | s_t)}{\pi_{\text{ref}}(y_t | s_t)} \quad (2)$$

Here the scalar RM reward is granted only at the final token ($t = T$), while every token pays a KL toll equal to β times its log-probability ratio. Summing $-\log(\pi_\theta/\pi_{\text{ref}})$ over tokens sampled from π_θ is an unbiased estimate of $-\text{KL}(\pi_\theta \| \pi_{\text{ref}})$, recovering Eq. (1).

3 Worked example: KL on a 3-token vocabulary

Let the vocabulary be $\{A, B, C\}$ with

$$\pi_\theta = (0.6, 0.3, 0.1), \quad \pi_{\text{ref}} = (0.5, 0.25, 0.25).$$

The per-token KL contributions $\pi_\theta(v) \log \frac{\pi_\theta(v)}{\pi_{\text{ref}}(v)}$:

$$A : 0.6 \log \frac{0.6}{0.5} = 0.6(0.182322) = 0.109393, \quad B : 0.3 \log \frac{0.3}{0.25} = 0.054696,$$

$$C : 0.1 \log \frac{0.1}{0.25} = -0.091629.$$

$$\text{KL}(\pi_\theta \| \pi_{\text{ref}}) = 0.109393 + 0.054696 - 0.091629 = \boxed{0.072460} \text{ nats.}$$

Note one contribution is *negative* (token C , which the policy down-weighted), yet the total KL is non-negative — as it must be by Gibbs’ inequality.

3.1 Sweeping β : the effective reward

Suppose the response earns base reward $r = 1.2$. The KL-adjusted (effective) objective contribution is $r - \beta \text{KL}$:

β	$\beta \cdot \text{KL}$	$r - \beta \text{KL}$	comment
0.0	0.000000	1.200000	unconstrained (hacking risk)
0.1	0.007246	1.192754	gentle leash
0.5	0.036230	1.163770	moderate
1.0	0.072460	1.127540	tight leash

As β grows, more of the raw reward is “taxed away” by divergence from the reference, so the policy is only rewarded for moves that improve r by *more* than the KL cost.

3.2 Token-level shaped reward

If the sampled token is A , its log-ratio is $\log(0.6/0.5) = 0.182322$. The shaped per-token penalty from Eq. (2) is:

$$\beta = 0.1 : R = 1.2 - 0.1(0.182322) = 1.181768, \quad \beta = 1.0 : R = 1.2 - 1.0(0.182322) = 1.017678.$$

Tokens the policy has *inflated* relative to the reference (log-ratio > 0) are penalised; tokens it suppressed would receive a small bonus.

4 Heatmap: π_θ vs. π_{ref}

The two distributions over $\{A, B, C\}$ side by side; shade scales with probability mass.

	A	B	C
π_θ	0.60	0.30	0.10
π_{ref}	0.50	0.25	0.25

The policy has concentrated mass on A and stripped it from C ; the KL of 0.0725 nats quantifies exactly this reshaping.

5 PPO’s clipped surrogate objective

We cannot do exact policy gradient on J , so PPO maximises a *clipped surrogate* using a batch collected from the slightly older policy π_{old} . Define the probability ratio and the advantage A (how much better the action was than the baseline value):

$$\rho = \frac{\pi_\theta(y_t | s_t)}{\pi_{\text{old}}(y_t | s_t)}.$$

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E} \left[\min(\rho A, \text{clip}(\rho, 1 - \epsilon, 1 + \epsilon) A) \right] \quad (3)$$

The min with the clipped ratio caps how much a single update can change the policy: if a good action ($A > 0$) already has $\rho > 1 + \epsilon$, the gradient is switched off, preventing destructively large steps. The advantage here uses the shaped reward of Eq. (2), so the KL leash flows through PPO.

5.1 One clip step, $\epsilon = 0.2$

Take advantage $A = 1.5$ (a good action), then $A = -1.5$ (a bad action):

A	ρ	ρA	$\text{clip}(\rho) A$	$\mathcal{L}^{\text{CLIP}} = \min$	effect
+1.5	0.7	1.050	1.200	1.050	gradient flows
+1.5	1.0	1.500	1.500	1.500	at π_{old}
+1.5	1.3	1.950	1.800	1.800	clipped (capped)
-1.5	0.7	-1.050	-1.200	-1.200	clipped
-1.5	1.0	-1.500	-1.500	-1.500	at π_{old}
-1.5	1.3	-1.950	-1.800	-1.950	gradient flows

For $A > 0$, $\rho = 1.3$: $\text{clip}(1.3, 0.8, 1.2) = 1.2$, so the surrogate is capped at $1.2 \times 1.5 = 1.8$ instead of 1.95 — the policy gains nothing from pushing ρ further, so the update halts. For $A < 0$, clipping kicks in on the *other* side ($\rho < 1 - \epsilon$), again removing the incentive to overshoot.

PPO optimizes a *pessimistic* (lower-bound) surrogate: the min always takes the less favourable of clipped and unclipped objectives. This trust-region-like behaviour keeps each update small, while the KL penalty in the reward keeps the *cumulative* policy near π_{ref} .

6 Properties that matter

1. **KL is non-negative** and zero iff $\pi_{\theta} = \pi_{\text{ref}}$; it is the price of moving away from the reference.
2. **β controls the reward/fidelity trade-off**; InstructGPT also adapts β to hit a target KL.
3. **Clipping removes the incentive to overstep** but does not hard-constrain ρ ; the gradient simply vanishes in the clipped region.
4. **Heavy machinery.** PPO needs the policy, the frozen reference, the reward model, and a value head — four networks. Topic 3 (DPO) collapses this to one loss.

The KL leash is one-sided in spirit: PPO uses the *forward-ish* sampling KL $\text{KL}(\pi_{\theta} \parallel \pi_{\text{ref}})$, estimated from policy samples. A common bug is estimating it as the bare $-\log(\pi_{\theta}/\pi_{\text{ref}})$ without realising this is an unbiased but high-variance estimator; the squared/exponential “ k_3 ” estimator is preferred. Also: β too small $\Rightarrow$ reward hacking; too large $\Rightarrow$ no learning. It must be tuned.

7 Where this sits in the track

	Trains against	Networks needed
Topic 1 (RM)	human preferences	1 (reward)
Topic 2 (PPO)	reward $-\beta$ KL	4 (policy, ref, RM, value)
Topic 3 (DPO)	preferences directly	2 (policy, ref)

A Reproducible (NumPy)

Inputs: $\pi_\theta = (0.6, 0.3, 0.1)$, $\pi_{\text{ref}} = (0.5, 0.25, 0.25)$, base reward $r = 1.2$, clip $\epsilon = 0.2$, advantage $A = \pm 1.5$. Key outputs: $\text{KL} = 0.072460$; effective reward at $\beta = 1$ is 1.127540; clipped surrogate at $\rho = 1.3$, $A = 1.5$ is 1.800.

```
import numpy as np

pt = np.array([0.6, 0.3, 0.1])    # pi_theta
pr = np.array([0.5, 0.25, 0.25]) # pi_ref
KL = np.sum(pt * np.log(pt / pr))
print("KL =", KL)                 # 0.07246032792714363

r = 1.2
for beta in [0.0, 0.1, 0.5, 1.0]:
    print(beta, r - beta * KL)    # 1.2, 1.192754, 1.163770, 1.127540

# PPO clip, one step
eps = 0.2
for A in (1.5, -1.5):
    for rho in (0.7, 1.0, 1.3):
        c = np.clip(rho, 1 - eps, 1 + eps)
        print(A, rho, min(rho * A, c * A))
# A=1.5: 1.05, 1.5, 1.8 ; A=-1.5: -1.2, -1.5, -1.95
```